

AI Algorithms Guide

A file-by-file explanation of the Python programs in the AI laboratory folders.

This guide describes the algorithms as implemented in the source files and calls out important implementation limitations where they affect

Contents: lab1 (6 files), lab2 (2), lab3 (2), lab4 (2), lab5 (2), lab7 (2).

lab1 - Introductory Search and Games

parking.py

Purpose: Reads a parking grid with E (entrance), R (road), A (available), and blocked cells, then finds the nearest reachable parking space.

Algorithm: Breadth-first search (BFS) on a four-neighbor grid. The entrance is placed in a FIFO queue. Each reachable R or A cell is visited once and its predecessor is recorded. The first A cell removed from the queue is the nearest in number of moves; parent pointers reconstruct the route.

Complexity / notes: Time and space: $O(\text{rows} * \text{cols})$. This is the complete working parking search in lab1.

parkingsim.py

Purpose: Attempts the same nearest-parking problem as an interactive simulation.

Algorithm: Intended BFS with parent-pointer route reconstruction over four neighboring cells.

Complexity / notes: Intended complexity: $O(\text{rows} * \text{cols})$ time and space. As written, it uses for i in rows instead of range(rows), assigns into an uninitialized grid, and may use target before assignment, so it is a broken draft rather than an executable implementation.

tic-tac-engine.py

Purpose: Runs human-versus-AI Tic-Tac-Toe; the human is X and the AI is O.

Algorithm: Minimax exhaustive game-tree search. Terminal states score O win = 1, X win = -1, draw = 0. The AI maximizes its score while the simulated human minimizes it, and best_move evaluates every legal AI move.

Complexity / notes: Worst-case work is approximately $O(9!)$ without memoization; recursion space is $O(9)$. The AI is optimal for the scoring model.

tic-tac-toe.py

Purpose: Reads a 3 by 3 board and reports X or O as winner, otherwise Draw.

Algorithm: Direct rule-based scan of all rows, columns, and both diagonals. A line counts only when its symbol is not the underscore empty marker.

Complexity / notes: For the fixed board: $O(1)$ time and $O(1)$ extra space. It assumes well-formed input and does not detect contradictory multiple-winner boards.

water-jug-gen.py

Purpose: Finds operations that reach a requested pair of jug volumes.

Algorithm: BFS over a finite state space (x, y). For every state it generates fill, empty, and both pour operations. A visited set prevents repetition; parent and action dictionaries reconstruct the solution.

Complexity / notes: At most capacity1 * capacity2 volume states, with the same order of space. The example searches capacities 4 and 3 from (0, 0) to (2, 0).

water-jug.py

Purpose: Demonstrates one fixed way to put exactly two liters in the 4-liter jug using a 3-liter jug.

Algorithm: Manual state-transition simulation: fill, pour, refill, transfer, and empty operations are explicitly assigned and printed.

Complexity / notes: $O(1)$ for this fixed sequence. It is not a general solver and does not search alternatives.

lab2 - Agents and Cost-Based Navigation

agent.py

Purpose: Compares a simple cleaning agent with a memory-based agent on a grid containing S, D, and X cells.

Algorithm: Both use queue-based exploration. `simple_agent` enqueues every non-obstacle neighbor, allowing repeated visits and stopping after 100 iterations. `memory_agent` marks cells when enqueued, so each reachable cell is explored at most once. D cells become C when cleaned.

Complexity / notes: Memory version: $O(\text{rows} * \text{cols})$ space and neighbor work. The simple version can duplicate queue entries. Both mutate the same grid, so the second agent receives the first agent's changes; its printed dirty value is the remaining argument, not the number cleaned.

dij.py

Purpose: Compares a goal-based delivery agent with a utility-based agent from S through P to G.

Algorithm: Goal-based search uses BFS for minimum movements. Utility-based search uses Dijkstra separately for S-to-P and P-to-G. Cell cost is 0.4 distance + 0.3 energy + 0.2 risk + 0.1 traffic. Parent pointers reconstruct routes.

Complexity / notes: BFS is $O(V + E)$. Heap-based Dijkstra is $O((V + E) \log V)$ per stage.

Utility search can use more moves to avoid expensive cells.

lab3 - Graph Search

Bidirection_search.py

Purpose: Finds a path between two nodes in an undirected, interactively entered graph.

Algorithm: Bidirectional BFS. One FIFO search grows from the source and another from the goal. When a newly discovered node appears in the other visited set, parent chains from the meeting point are joined.

Complexity / notes: $O(V + E)$ time and $O(V)$ space for an unweighted graph, often with fewer expansions than one-sided BFS.

DLS.py

Purpose: Solves an 8-puzzle only up to a user-supplied depth limit.

Algorithm: Depth-limited DFS with backtracking. The blank tile is swapped with each legal neighbor. Recursion stops at the goal or depth limit. A path-local visited set prevents cycles, then removes failed states during backtracking.

Complexity / notes: Worst case $O(b^d)$ time and $O(bd)$ space, where b is branching factor and d is the depth limit. Reports recursive calls as generated nodes.

lab4 - 8-Puzzle Search Comparison

8_puzzle.py

Purpose: Compares BFS, depth-limited DFS, and A* on user-provided start and goal states.

Algorithm: BFS stores full paths in a FIFO queue. DFS recursively explores up to `depth_limit`. A* orders states by $f(n) = g(n) + h(n)$, where h counts nonblank tiles that do not match the goal. Each move swaps the blank with a legal neighbor.

Complexity / notes: BFS is exponential in solution depth and memory. DFS is $O(b^d)$ time with $O(d)$ depth. A* is exponential worst case but is guided by the heuristic. Full paths are stored in queue entries.

shortest_path.py

Purpose: Finds a route from a fixed weighted graph node a to f .

Algorithm: Dijkstra uses a min-heap keyed by known cost $g(n)$. A* uses $f(n) = g(n) + h(n)$, with the supplied heuristic, while both relax edges and store parent pointers.

Complexity / notes: With a binary heap, $O((V + E) \log V)$ time and $O(V)$ space. A* is optimal when its heuristic is admissible and consistent.

lab5 - Local Optimization

hill_climb.py

Purpose: Finds a short route from warehouse W through delivery locations A-G and back.

Algorithm: Steepest-ascent hill climbing on route permutations. A neighbor is made by swapping two positions. All 21 swaps are scored; the best improving neighbor becomes the next route. It stops at a local optimum.

Complexity / notes: About $O(n^3)$ per iteration because $O(n^2)$ neighbors each cost $O(n)$ to score; route storage is $O(n^2)$ during evaluation. Random initialization means results vary and local minima are possible.

annealing.py

Purpose: Searches for a low-distance delivery route with randomized exploration.

Algorithm: Simulated annealing. A random swap neighbor is always accepted when better; a worse neighbor is accepted with probability $\exp(-\Delta / \text{temperature})$. Temperature cools geometrically from 100 by 0.95 until below 0.1, while the best route seen is retained.

Complexity / notes: For k iterations, about $O(k \cdot n)$ time to score routes and $O(n)$ route space. It can escape local minima but produces stochastic results.

lab7 - Genetic Algorithms

Q1.py

Purpose: Maximizes $f(x) = 15x - x^2$ for integer x in $[0, 15]$.

Algorithm: Binary genetic algorithm. Four-bit chromosomes encode x . Each generation uses roulette-wheel selection, one-point crossover, bit mutation at rate 0.05, and elitism that preserves the best chromosome.

Complexity / notes: For population P , generations G , and chromosome length L : about $O(G \cdot P \cdot L)$ time and $O(P \cdot L)$ space. The exact integer maximum is 56 at $x = 7$ or $x = 8$, although a random run is not guaranteed to find it.

Q2.py

Purpose: Solves a 0/1 knapsack with capacity 15 kg for items A-D.

Algorithm: Binary genetic algorithm. Each bit selects one item. Fitness sums values for feasible chromosomes and returns zero when weight exceeds capacity. Roulette selection, one-point crossover, mutation, and elitism evolve 25 generations.

Complexity / notes: About $O(G \cdot P)$ time and $O(P)$ space for four items. The exact optimum is 1110: A+B+C, weight 10 kg and value Rs. 1600. A stochastic run can still return another feasible result.

Algorithm themes

BFS is used when every move has equal cost and shortest depth is desired. Dijkstra handles nonnegative weighted costs. A* adds a heuristic to focus search.

DFS and depth-limited search trade completeness for lower memory. Hill climbing and simulated annealing optimize a route without enumerating the full state space.

Genetic algorithms evolve populations through selection, crossover, mutation, and elitism. The collection moves from basic state-space search to informed search and stochastic optimization.

Important distinction: a program can demonstrate an algorithm without being production-ready. parking.py, water-jug-gen.py, dij.py, the 8-puzzle implementations, lab5, and lab7 are the clearest working demonstrations. parkingsim.py contains input and initialization errors, and several interactive programs assume well-formed input.